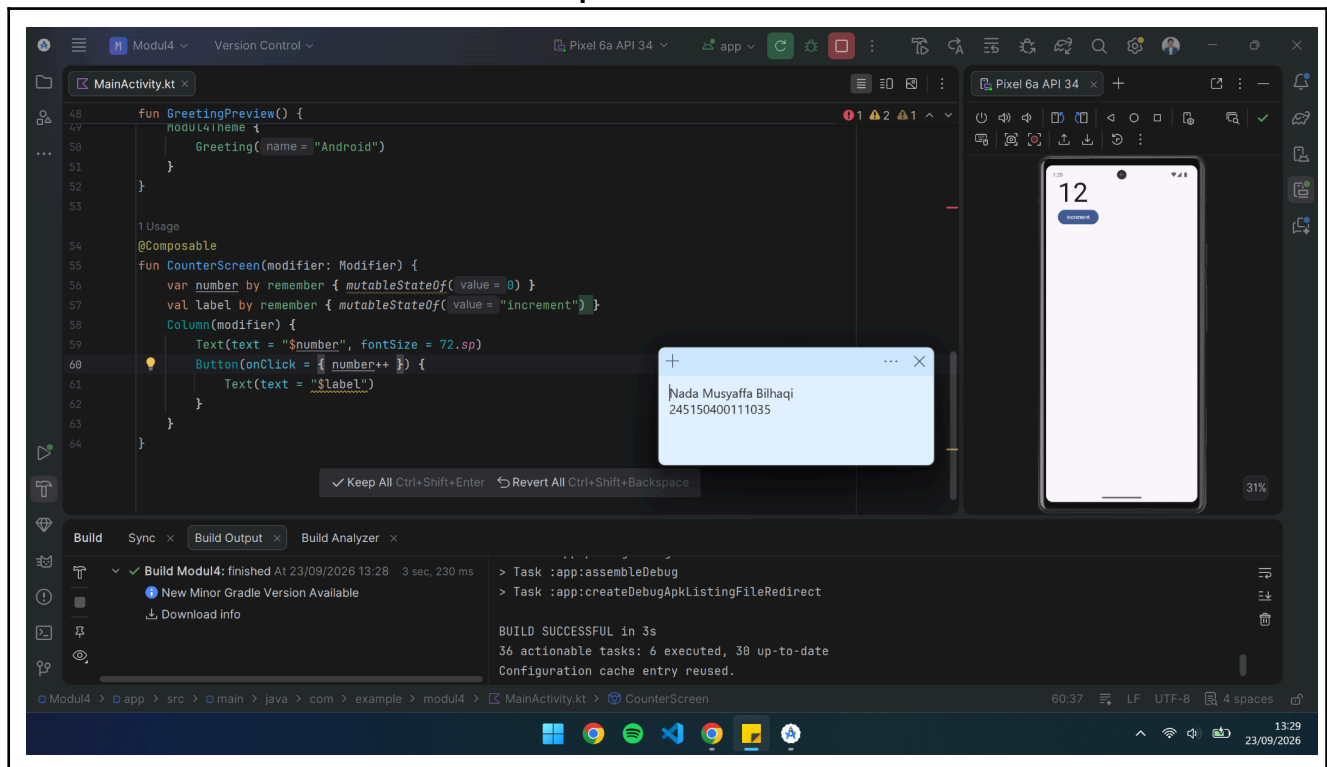


Laporan Praktik Modul 4 Pemrograman Aplikasi Mobile

Nama	Nada Musyaffa Bilhaqi
NIM	245150400111035
Program Studi	Sistem Informasi
Kelas	PAM-C

STATE HOISTING

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik... ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.



2. Apa yang terjadi setiap kali tombol "Increment" diklik?

Angka yang ditampilkan pada layar bertambah 1 setiap kali tombol tersebut diklik.

3. Apa yang menyebabkan hal tersebut terjadi?

Hal ini terjadi karena variabel number menggunakan mutableStateOf (sebagai state), sehingga saat event onClick = { number++ } dijalankan, nilai state tersebut berubah. Perubahan state memicu recomposition pada Jetpack Compose, yang otomatis menggambar ulang elemen UI dengan angka terbaru.

4. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?

Variabel state number.

5. Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?

Variabel state label.

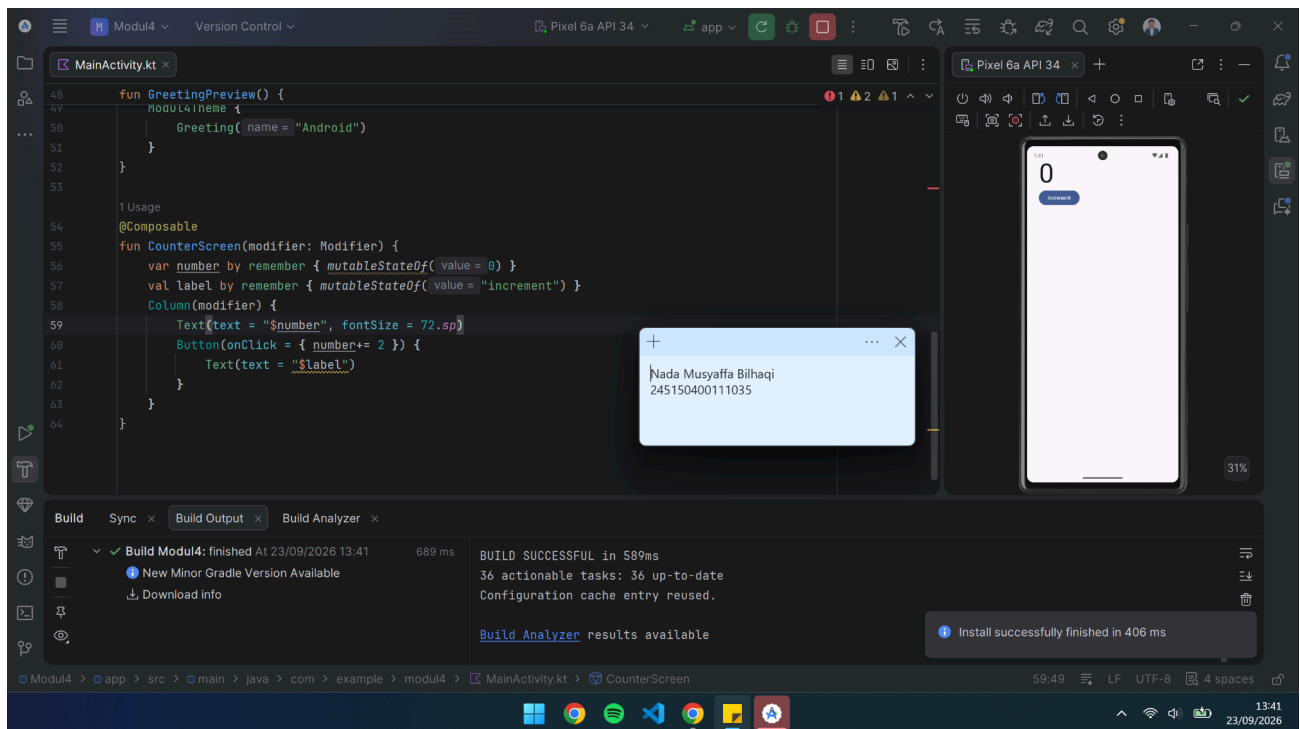
6a. Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2... Perubahan apa yang perlu dilakukan dan dilakukan di mana?

Perubahan harus dilakukan secara langsung di dalam body fungsi CounterScreen.

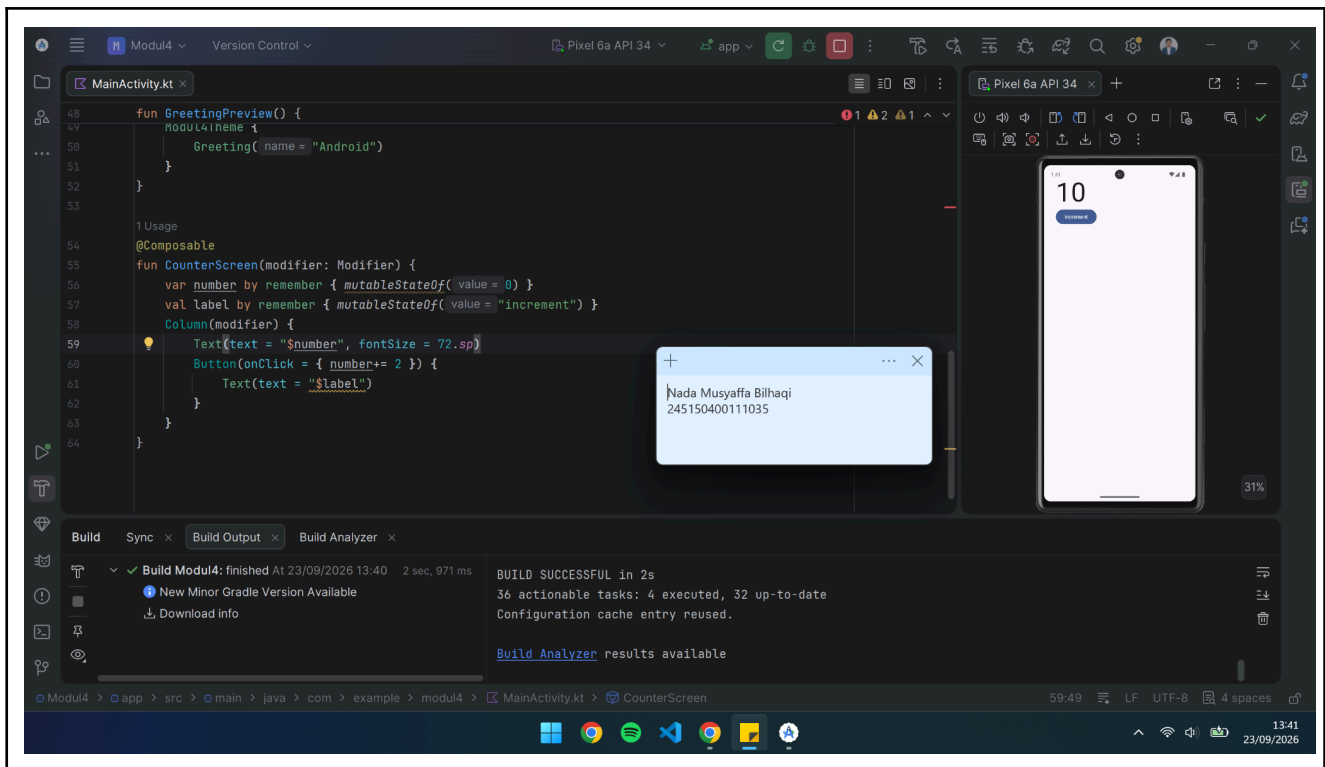
- Ubah nilai label menjadi: `val label by remember { mutableStateOf("Increment 2") }`
- Ubah event onClick pada Button menjadi: `Button(onClick = { number += 2 })`

6b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

Sebelum



Sesudah



7a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap sebelumnya?

Ya, program tetap berjalan dengan baik dan menampilkan visual fungsional yang sama persis.

7b. Di bagian kode program manakah saat ini proses increment dilakukan?

Proses increment sekarang dilakukan di bagian komponen induk di dalam fungsi onCreate(), tepatnya pada argumen fungsi callback onClick = { number++ }

7c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

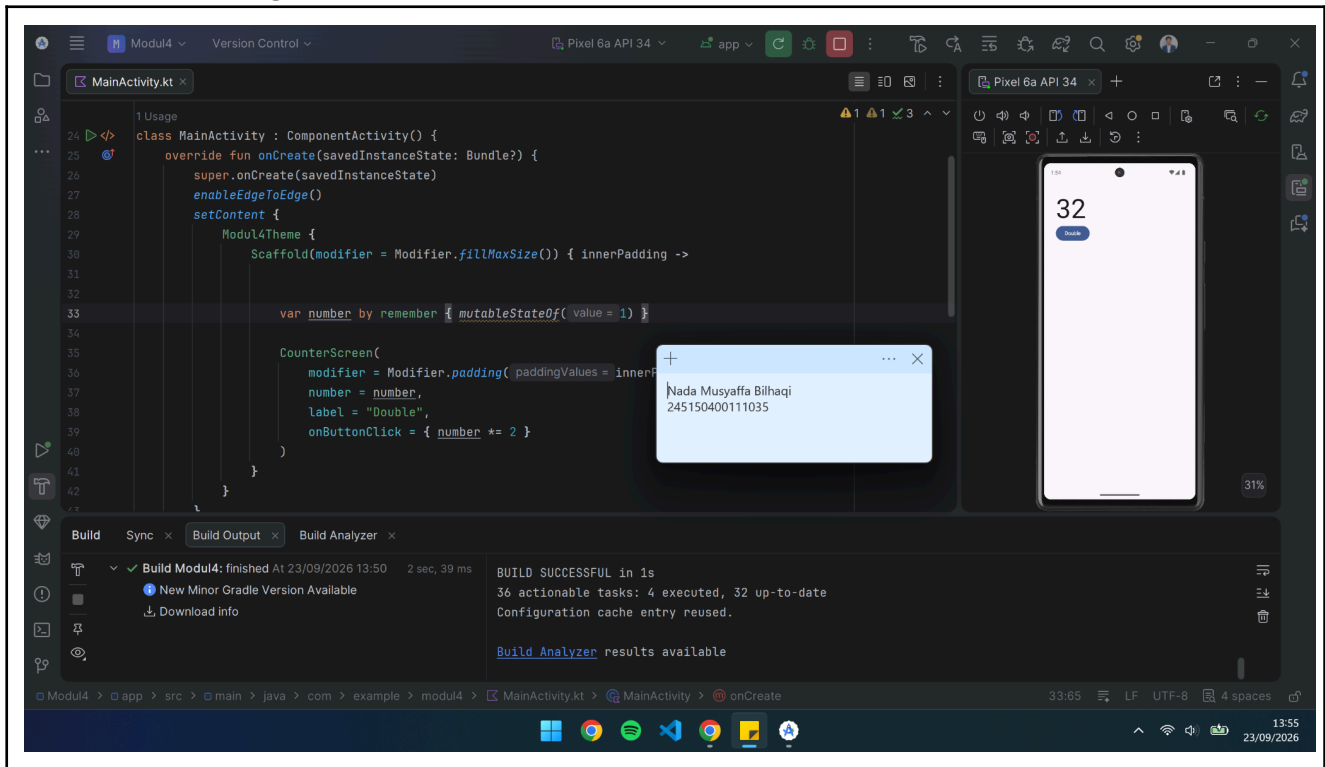
State number sekarang dideklarasikan di dalam fungsi onCreate() sebelum melakukan pemanggilan fungsi Composable CounterScreen. Untuk label teks, datanya sudah tidak lagi menggunakan state, melainkan diteruskan langsung sebagai parameter tipe String

8d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

```
var number by remember { mutableStateOf(1) } // 8(b): Nilai awal diubah menjadi 1
```

```
CounterScreen(
    modifier = Modifier.padding(32.dp),
    number = number,
    label = "Double", // 8(a): Label menjadi "Double"
    onClick = { number *= 2 } // 8(c): Menggandakan nilai variabel number
)
```

9a. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.



9b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Seluruh perubahan logika dan nilai dilakukan di pemanggil composable induk (onCreate()), bukan pada file/fungsi composable UI-nya.

9c. Apakah ada perubahan yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan.

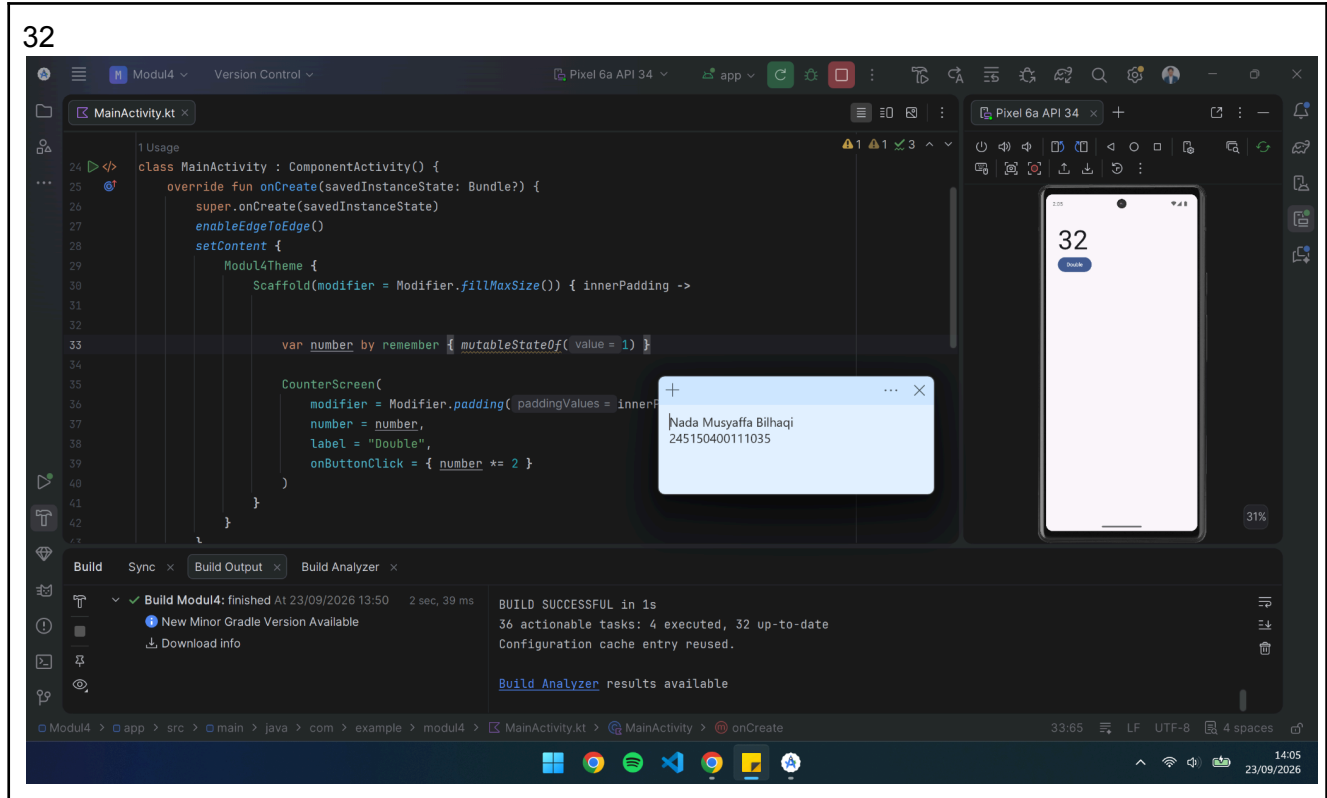
Tidak ada perubahan apa pun di dalam body fungsi CounterScreen. Komponen tersebut sudah bersifat stateless (tidak menyimpan data secara internal), sehingga murni hanya menerima data melalui parameter yang diberikan pemanggilnya.

10. Apa arti dari jawaban pada pertanyaan sebelumnya terkait dengan manfaat dari melakukan state hoisting?

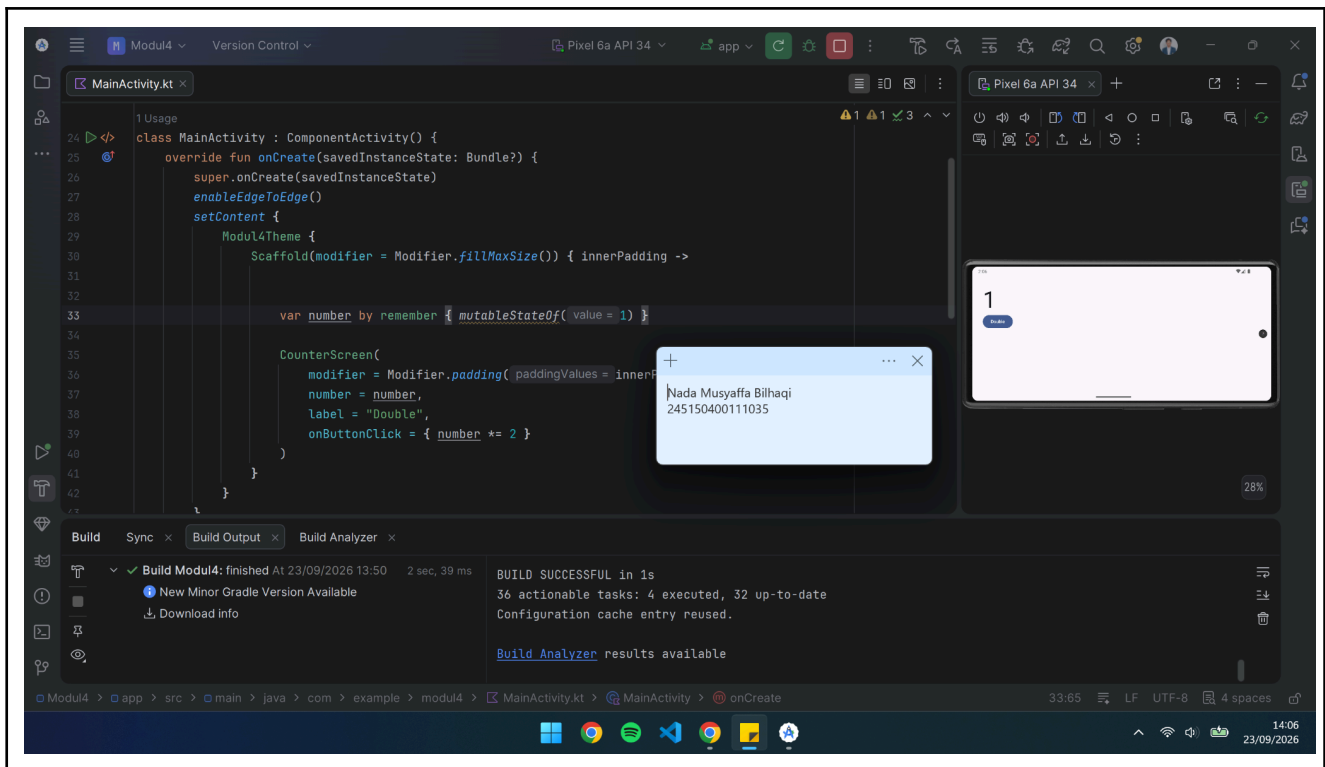
Jawaban tersebut menegaskan bahwa state hoisting berhasil memisahkan logika UI/presentasi dengan logika bisnis (manajemen data). Manfaat utamanya adalah composable (CounterScreen) menjadi reusable (dapat digunakan kembali) dan sangat mudah dimodifikasi kebutuhannya (dari sekadar menambah menjadi menggandakan angka) tanpa harus menyentuh apalagi merusak kode struktur antarmuka UI di dalamnya. Hal ini menjamin penerapan prinsip Single Source of Truth (SSoT) pada aplikasi.

PERUBAHAN KONFIGURASI

1. Jalankan program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol "Double"... Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



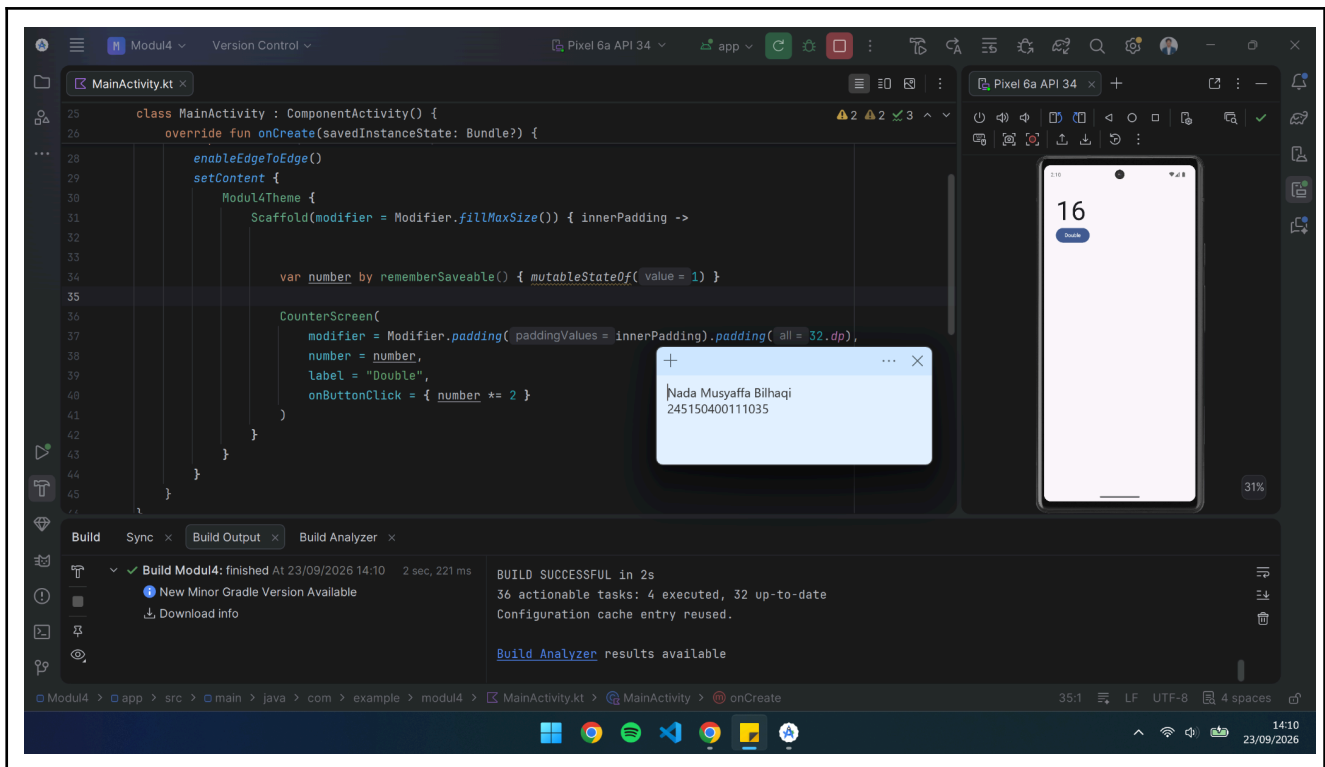
2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



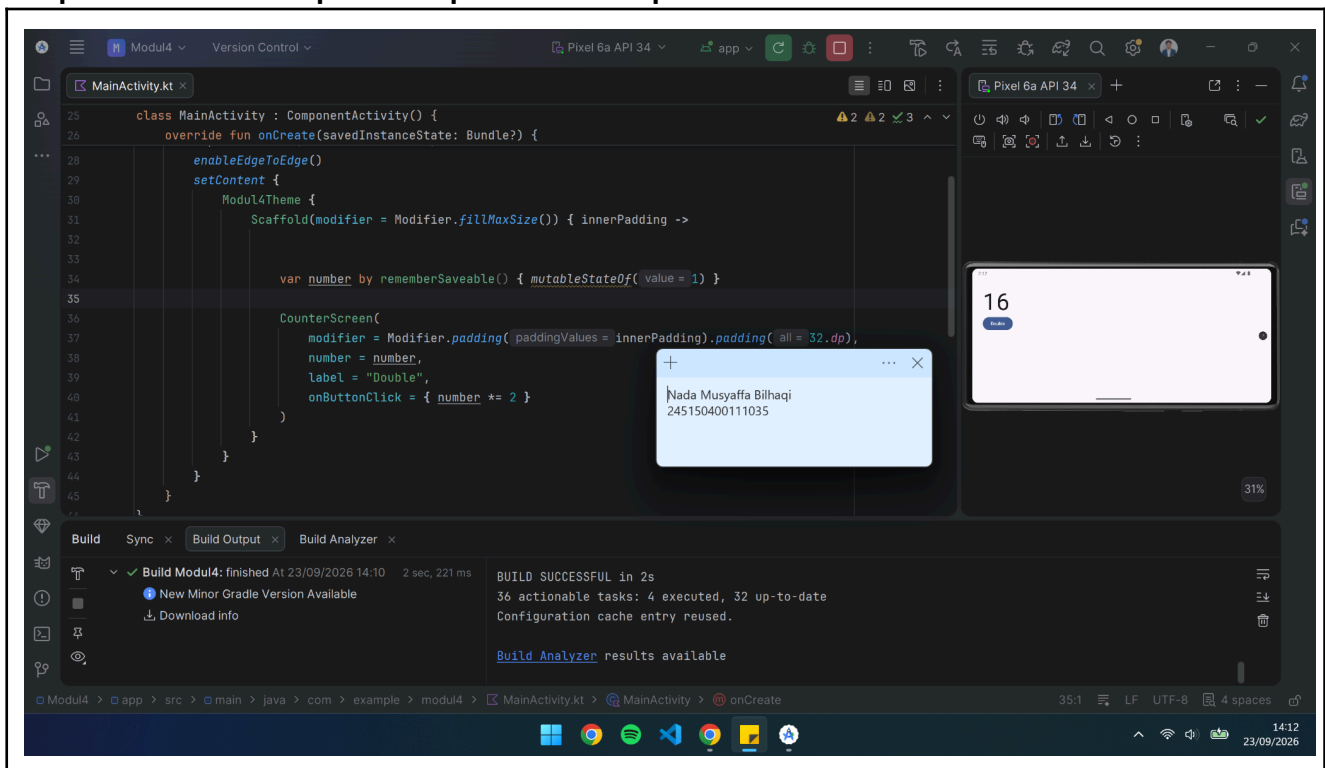
3. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan akan ter-reset kembali ke nilai awalnya, yaitu 1. Hal ini terjadi karena meskipun fungsi remember menjaga status agar tetap ada saat terjadi proses recomposition, namun nilai yang disimpan oleh remember akan hilang jika Activity dihancurkan, contohnya saat terjadi rotasi layar perangkat. Saat diputar, Android menghancurkan dan membuat ulang Activity, sehingga state kembali ke nilai default.

5. Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol "Double"... Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



6. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape... Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



7. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan tetap yakni 16. Hal ini terjadi karena sudah menggunakan

rememberSaveable yang secara otomatis menyimpan nilai ke dalam objek Bundle. Data yang disimpan di dalamnya akan tetap aman meskipun terjadi perubahan konfigurasi seperti rotasi layar, atau jika sistem mematikan aplikasi untuk menghemat memori (RAM).

8. Apa fungsi dari konstruk rememberSavable pada kode program Android yang membedakannya dari konstruk remember?

rememberSaveable berperilaku mirip seperti remember di mana ia mempertahankan status di seluruh recomposition. Namun pembedanya adalah, rememberSaveable juga mempertahankan status tersebut di seluruh pembuatan ulang Activity atau proses menggunakan mekanisme status instance yang tersimpan (Saved Instance State / Bundle).

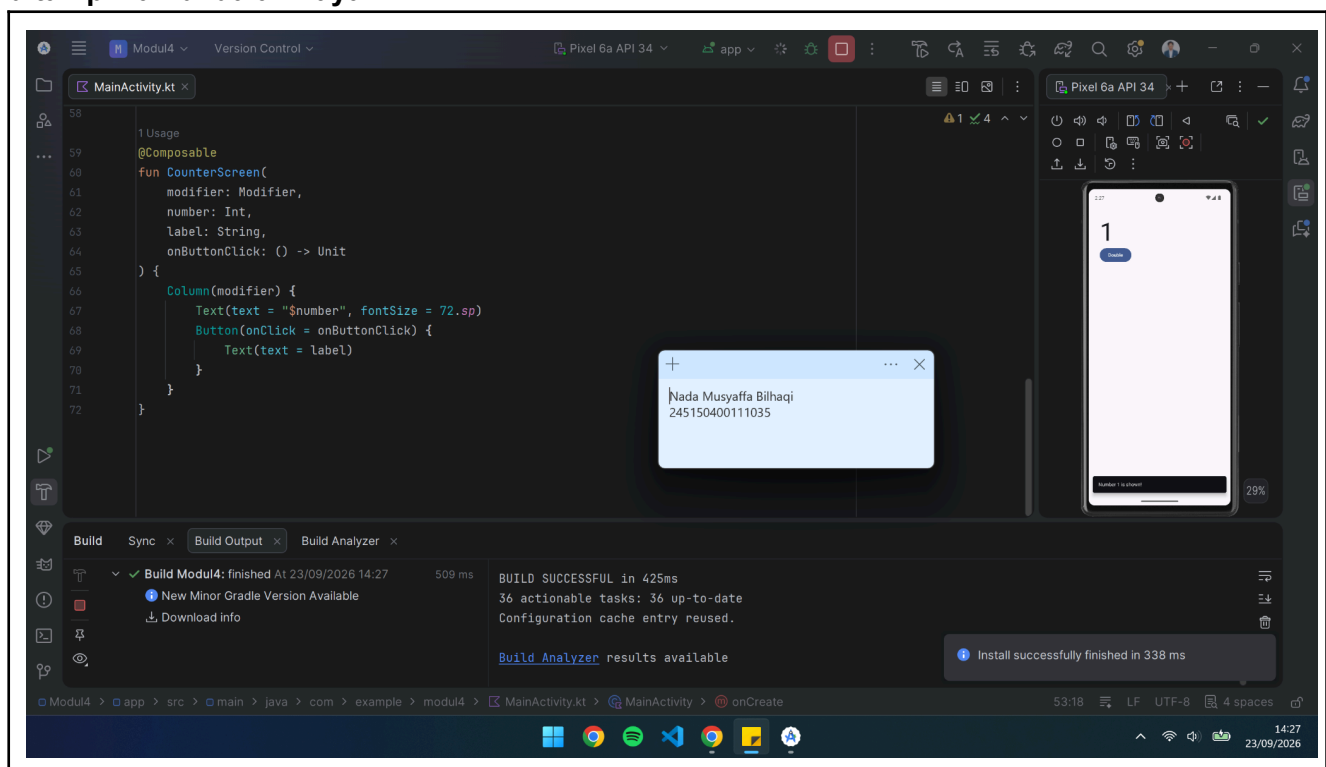
9. Beri contoh kasus nyata lain di mana penggunaan konstruk rememberSavable ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Contoh kasus nyata: Mengamankan nilai input teks di dalam form pendaftaran pengguna (seperti nama, email, password), status checkbox yang sudah dicentang oleh pengguna, atau posisi nilai scroll pada sebuah daftar (list) panjang.

Manfaat penerapannya: Memastikan pengalaman pengguna (User Experience) yang baik, di mana data yang sudah diisi atau interaksi yang sudah dilakukan oleh pengguna tidak hilang dengan sia-sia hanya karena pengguna tidak sengaja memutar orientasi layar perangkat atau ketika mereka meminimalisasi aplikasi untuk membuka aplikasi lain.

LAUNCHED EFFECT-1

1a. Pada saat pertama kali program dijalankan, tanpa mengklik button “Double” apa yang ditampilkan di dalam layar?



Pada bagian bawah layar aplikasi, akan muncul sebuah Snackbar (notifikasi pop-up singkat) yang menampilkan pesan teks "Number 1 is shown!". Angka 1 di tengah layar juga ditampilkan seperti biasa.

1b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

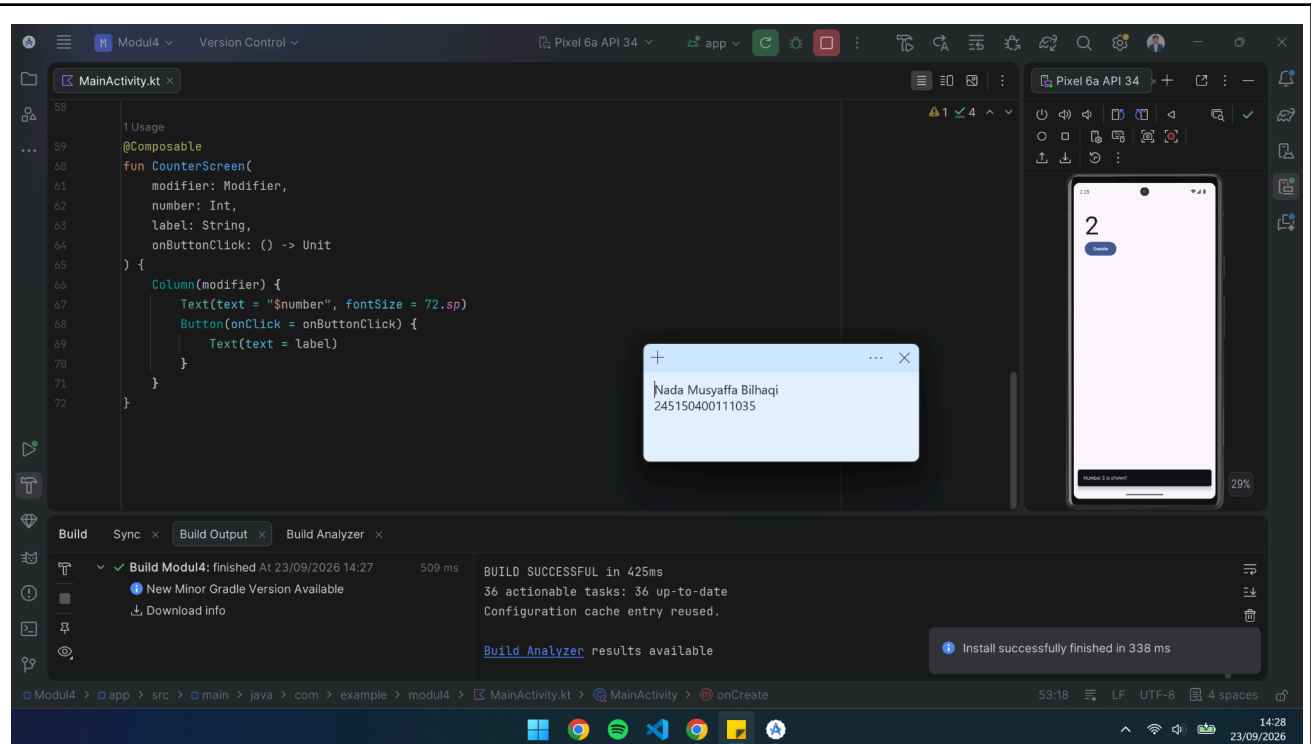
Hal ini terjadi karena cara kerja `LaunchedEffect`. Fungsi side-effect `LaunchedEffect` dirancang untuk secara otomatis menjalankan blok kode asinkron yang ada di dalamnya pada saat komponen UI (`Composable`) pertama kali dimuat atau masuk ke dalam komposisi (`enters the composition`). Oleh karena itu, tanpa perlu diklik, pesan tersebut akan langsung muncul saat aplikasi terbuka.

1c. Apa yang memicu pesan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Pesan tersebut dipicu oleh pemanggilan fungsi `snackbarHostState.showSnackbar()` yang dibungkus di dalam blok `LaunchedEffect`. Bagian kode program yang memicunya adalah:

```
LaunchedEffect(number) {  
    snackbarHostState.showSnackbar("Number $number is shown!")  
}
```

2a. Klik button “Double” satu kali, Apa yang terjadi?



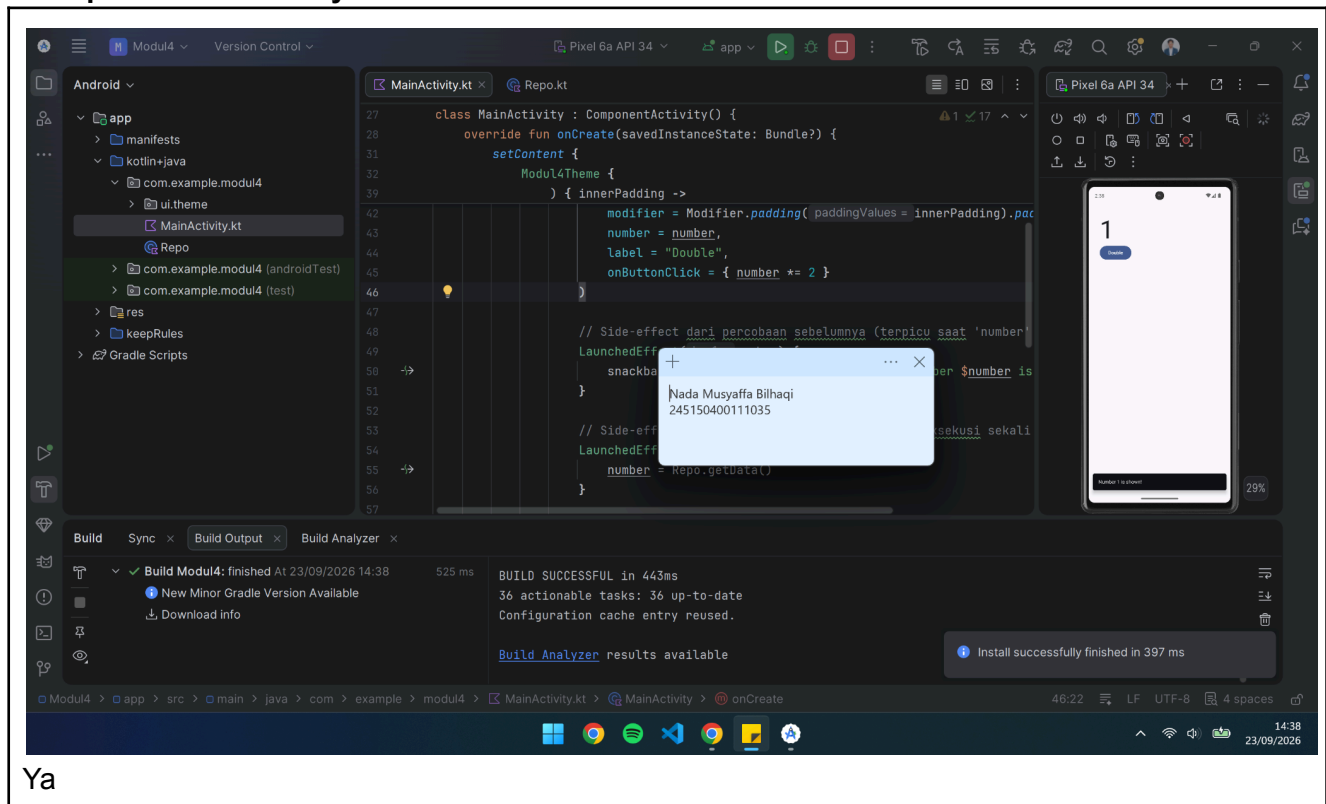
Angka utama pada antarmuka aplikasi akan berubah menjadi 2 (karena dikali 2). Bersamaan dengan itu, akan muncul pesan Snackbar baru di bagian bawah layar dengan tulisan "Number 2 is shown!".

2b. Mengapa hal tersebut terjadi? Jelaskan!

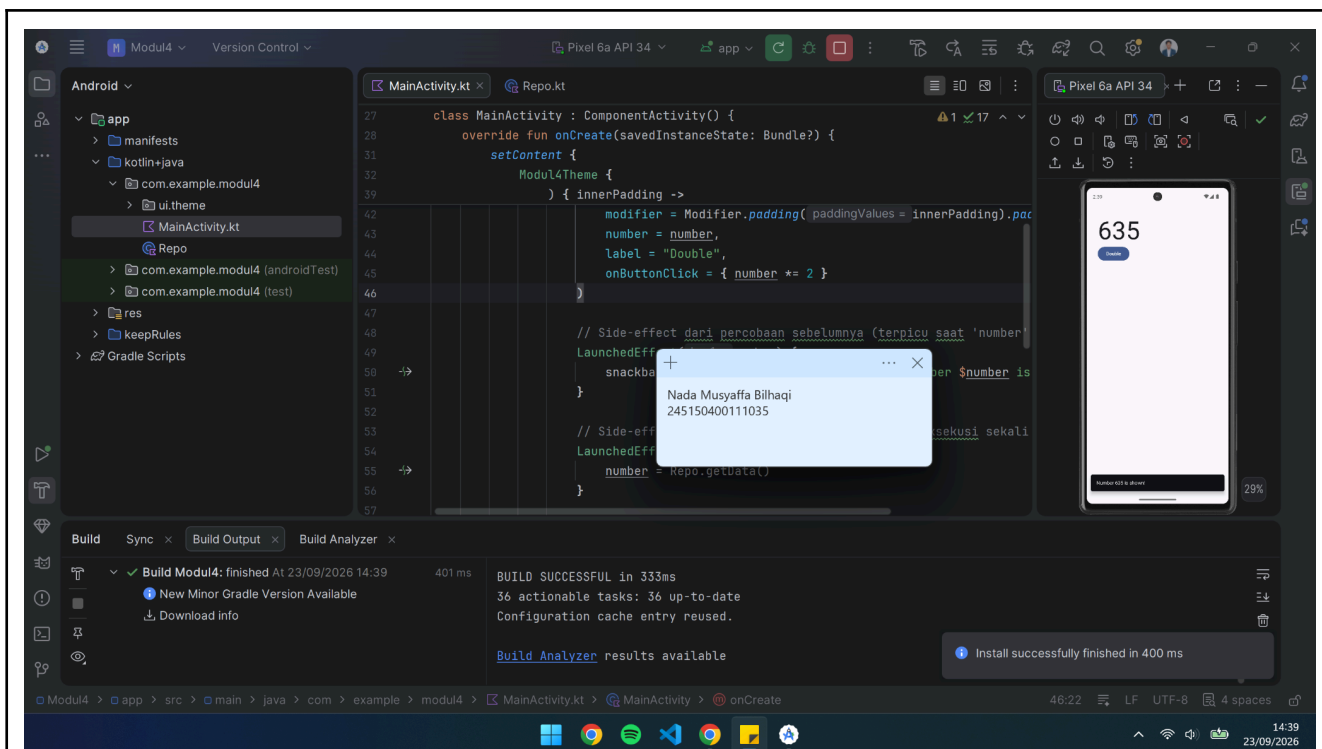
Hal tersebut terjadi karena variabel state `number` dipasang sebagai parameter key (kunci) pada fungsi `LaunchedEffect(number)`. Ketika tombol "Double" diklik, nilai variabel `number` berubah nilainya (dari 1 menjadi 2). Berdasarkan konsep kerjanya, jika key yang diberikan mengalami perubahan, `LaunchedEffect` akan secara otomatis membatalkan proses yang lama dan mengeksekusi ulang/kembali blok kode program di dalamnya dengan menggunakan nilai state yang paling baru. Akibatnya, `showSnackBar` dipanggil lagi dengan menyertakan nilai `number` terbaru.

PERUBAHAN KONFIGURASI 2

1a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?



1b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?



Setelah dua detik, angka "1" di layar akan tiba-tiba berubah menjadi angka acak (berkisar antara 100 hingga 999). Bersamaan dengan itu, *SnackBar* baru akan muncul menampilkan pesan yang sesuai dengan angka acak tersebut, misal pada gambar **"Number 635 is shown!"**.

1c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada *SnackBar* tersebut ditampilkan? Pada bagian kode program yang mana?

- Perubahan angka dipicu oleh fungsi asinkron yang mengembalikan angka acak setelah penundaan 2 detik. Ini terjadi pada bagian kode:

```
LaunchedEffect(Unit) {
    number = Repo.getData() // Memperbarui state 'number'
}
```

- Pesan *SnackBar* baru terpicu karena Jetpack Compose mendeteksi bahwa state number telah berubah nilainya. Perubahan ini menyebabkan `LaunchedEffect(number)` yang menjadikan number sebagai key ikut tereksekusi ulang dengan nilai terbarunya. Ini terjadi pada bagian kode:

```
LaunchedEffect(number) { // Key mendeteksi perubahan nilai
    snackbarHostState.showSnackbar("Number $number is shown!")
}
```

2a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

- `LaunchedEffect(state)` menggunakan variabel dinamis (state) sebagai key parameternya. Jika nilai dari key tersebut berubah, sistem akan membatalkan proses coroutine yang sedang berjalan dan menjalankannya kembali dari awal dengan data yang baru.
- `LaunchedEffect(Unit)` menggunakan objek konstan Unit (yang nilainya tidak akan pernah berubah) sebagai parameternya. Akibatnya, blok kode di dalamnya hanya akan dieksekusi

satu kali saja seumur hidup komponen tersebut, dan tidak akan pernah di-restart ulang meskipun terjadi recomposition berkali-kali.

2b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

- `LaunchedEffect(state)` dijalankan pada saat pertama kali Composable masuk ke dalam komposisi layar (UI dimuat), DAN setiap kali nilai dari variabel state yang menjadi key-nya mengalami perubahan.
- `LaunchedEffect(Unit)` dijalankan hanya satu kali secara eksklusif, yaitu saat pertama kali Composable masuk ke dalam komposisi layar (saat UI pertama kali dimuat ke memori).